SIMATS ENGINEERING

ASSESSMENT TOOL 3 - Output Prediction

COURSE NAME: Data Structures **COURSE CODE:** CSA03

Register Number	1972260011
Name	Dhavanesh S S

Assessment Tool Weightage: 35%

COURSE OUTCOME COVERED

CO1: *Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL3, BL4)*

Questions: 5 **EACH QUESTIONS CARRIES:** 5 MARKS **Total Marks:** 25 **Duration:** ONE HOUR

S. No.	List of Questions	

1	**Array Traversal Program**	`#include <stdio.h>`
		`int main()`
		`{`
		`  int a[5] = {10, 20, 30, 40, 50};`
		`  int i;`
		`  for(i = 0; i < 5; i++)`
		`  printf("%d ", a[i]);`
		`  return 0;`
		`}`

2	**Sum of Array Elements Program**	`#include <stdio.h>`
		`int main()`
		`{`
		`  int a[] = {2, 4, 6, 8};`
		`  int sum = 0, i;`
		
		`  for(i = 0; i < 4; i++)`
		`  sum += a[i];`
		
		`  printf("%d", sum);`
		`  return 0;`
		`}`

3	**Recursive Factorial Program**	`#include <stdio.h>`
		
		`int fact(int n)`
		`{`
		`  if(n == 0)`
		`  return 1;`
		`  return n * fact(n - 1);`
		`}`
		
		`int main()`
		`{`
		`  printf("%d", fact(4));`
		`  return 0;`
		`}`

4	**Recursive Fibonacci Program**	`#include <stdio.h>`
		
		`int fib(int n)`
		`{`
		`  if(n <= 1)`
		`  return n;`
		`  return fib(n-1) + fib(n-2);`
		`}`
		
		`int main()`
		`{`
		`  printf("%d", fib(6));`
		`  return 0;`
		`}`

5	**Function as ADT Operation Program**	`#include <stdio.h>`
		
		`int add(int x, int y)`
		`{`
		`  return x + y;`
		`}`
		
		`int main()`
		`{`
		`  printf("%d", add(15, 25));`
		`  return 0;`
		`}`

Code of Conduct:

I Dhavanesh S S / 1972260011 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (5 Marks)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (3 Marks)	Level 1 – Beginning (1–2 Marks)
Understanding of Program Logic	1	Correctly identifies the complete program flow and execution sequence.	Minor misunderstanding of program flow.	Partial understanding of logic.	Unable to interpret the program logic.
Trace of Variables / Function Calls	1	Accurately traces all variable values, iterations, and recursive calls.	Traces most values correctly with minor errors.	Traces some values but misses important steps.	Incorrect or no tracing.
Output Prediction Accuracy	2	Output predicted completely and correctly.	Output mostly correct with minor mistakes.	Output partially correct.	Output incorrect or missing.
Complexity Interpretation (Time/Space)	1	Correctly identifies and justifies complexity where applicable.	Correct complexity with limited justification.	Partial understanding of complexity.	Incorrect or no complexity analysis.

CO1 – AT3: ANSWERS

Each answer includes program logic, trace, output prediction and complexity.

1. Array Traversal Program

Program

```
#include <stdio.h>
int main() {
    int a[5] = {10, 20, 30, 40, 50};
    int i;
    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);
    return 0;
}
```

Understanding of Program Logic

The loop starts at i = 0 and runs while i < 5. Each iteration prints a[i]. The values printed are 10, 20, 30, 40 and 50.

Trace of Variables / Function Calls

i = 0 → 10; i = 1 → 20; i = 2 → 30; i = 3 → 40; i = 4 → 50; i = 5 stops the loop.

Output Prediction

Output: 10 20 30 40 50

Complexity Interpretation

Time: O(n), because every array element is visited. Auxiliary space: O(1).

2. Sum of Array Elements

Program

```
#include <stdio.h>
int main() {
    int a[] = {2, 4, 6, 8};
    int sum = 0, i;
    for (i = 0; i < 4; i++)
        sum += a[i];
    printf("%d", sum);
    return 0;
}
```

Understanding of Program Logic

The variable sum is initialized to 0. The loop adds each array element to sum.

Trace of Variables / Function Calls

Initially sum = 0; after 2, sum = 2; after 4, sum = 6; after 6, sum = 12; after 8, sum = 20.

Output Prediction

Output: 20

Complexity Interpretation

Time: O(n), because the loop processes n elements. Auxiliary space: O(1).

3. Recursive Factorial Program

Program

```
#include <stdio.h>
int fact(int n) {
    if (n == 0)
        return 1;
    return n * fact(n - 1);
}
int main() {
    printf("%d", fact(4));
    return 0;
}
```

Understanding of Program Logic

fact(n) calls fact(n - 1) until n reaches 0. The base case returns 1, and the pending calls multiply their values while returning.

Trace of Variables / Function Calls

fact(4) = 4 × fact(3) = 4 × 3 × fact(2) = 4 × 3 × 2 × fact(1) = 4 × 3 × 2 × 1 × fact(0) = 24.

Output Prediction

Output: 24

Complexity Interpretation

Time: O(n), as there are n recursive calls. Auxiliary space: O(n), due to the recursion stack.

4. Recursive Fibonacci Program

Program

```
#include <stdio.h>
int fib(int n) {
    if (n <= 1)
        return n;
    return fib(n - 1) + fib(n - 2);
}
int main() {
    printf("%d", fib(6));
    return 0;
}
```

Understanding of Program Logic

The base cases return 0 for fib(0) and 1 for fib(1). Each later term is the sum of the previous two terms.

Trace of Variables / Function Calls

fib(2)=1; fib(3)=2; fib(4)=3; fib(5)=5; fib(6)=8. The sequence is 0, 1, 1, 2, 3, 5, 8.

Output Prediction

Output: 8

Complexity Interpretation

Time: O(2^n) approximately for this simple recursive implementation. Auxiliary space: O(n) maximum recursion depth.

5. Function as ADT Operation

Program

```
#include <stdio.h>
int add(int x, int y) {
    return x + y;
}
int main() {
    printf("%d", add(15, 25));
    return 0;
}
```

Understanding of Program Logic

main() calls add(15, 25). The parameters x and y receive 15 and 25. The function returns their sum to printf.

Trace of Variables / Function Calls

Function call: add(15, 25); x = 15, y = 25; return x + y = 40; printf displays 40.

Output Prediction

Output: 40

Complexity Interpretation

Time: O(1), because a fixed number of operations is performed. Auxiliary space: O(1).

Code of Conduct

I Dhavanesh S S / 1972260011 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.